

Lab 3: grid dimensioning & data formatting
Geavanceerde Computerarchitectuur: Labosessie

(B-KUL-JLI1QS)

Timon Coen

March 3, 2024



Group: MEIswe

Instructor: Alex Marinšek

Introduction

In this lab session, we focus on optimisation techniques, using threads in a concise way and finding the best block size. To start, threads are used in groups of 32, called warps. If you assign threads to cores, you need to assign them per 32, so if 33 threads are needed, two warps or 64 cores are used. The GPU used in the lab (NVIDIA GeForce GT 740) has 192 cores in one stream multiprocessor (SM). This GPU has two of them. Within a block, only one SM can be used. If you want to use 384 cores, you better use $<<< 2, 192 >>$ in stead of $<<< 1, 384 >>>$. Because the first example will run on two SMs simultaneously and the second example will run on one SM sequentially. In this lab session we look at ways to minimise execution time based on block size and the amount of blocks that are used.

This is ideal in the case every thread has to do the same work. In some cases every other thread doesn't have to do a job or a smaller job, which makes those threads idle and useless, because all the threads in a warp finish together. This is called thread divergence. It could be solved by putting threads with the same magnitude of job in the same warp.

Other forms of thread divergence is striding. With striding, you can use iterations, like this one thread can do the work of three threads at once by using a for loop with three iterations. This can be useful if the limit is used and no parallel execution is possible. Like this no new cores need to be assigned, which will take also time. For example if you want to use 768 threads, in stead of using 4 blocks of 192, you can use 2 blocks of 192 and handle two jobs by every thread. Striding can both create or solve thread divergence. If you can do multiple small jobs per thread for a few threads and one large job each for the rest of the threads, the threads will not idle. On the other hand if only one thread in the warp does an extra iteration, all the other will have to wait. So here again, you need to see how you can use it in your favour.

Results

In table 1 and graph in figure 1 is visible that 192 cores excels at executing the fastest. This is as we expected, because the GT 740 has 192 available cores per block. Now we can look if striding helps. And we can clearly see that a higher number of iterations has a lower execution time. This is easy to explain, there are less blocks to be executed sequentially, but the sequencing is placed within the kernel, so less blocks are needed, less cores need to be initialised, which reduces time.

Next to this, we can see when we increase the block size from 192 to 193, the execution time doubles, because for every amount of blocks, there are two steps executed. First the first 192 threads are executed on the cores and thereafter the last one needs to be executed in another warp, so a lot more cores are allocated, resulting in a lot more execution time. As also visible in table 1 and figure 1.

In table 2 and figure 2 we can see the result of the implementation of colouring the picture red. Here again we see the best overall times for block size 192. For higher iterations, we see higher execution times, which can be caused by inconsistencies during measuring. We see a higher execution time than with the inversion. This is caused by thread divergence, because only the red value needs to be changed. All red values are in the same warp, but the ones before 100 and the ones after 200 that don't need to be changed, will need to wait until the operation is done, creating the thread divergence. A solution to counter thread divergence is to wisely adapt code, choose the right block size and put equal executions in the same warp or even full block.

Conclusion

The lessen learned here is that if you want to write optimal code for a specific GPU, it is worth it, to look how a GPU is constructed and what will work fastest. Using striding to your advantage, so less threads need to be initiated in sequence. Also look for thread divergence and try to execute equal operations in the same warps and blocks.

Table 1: Execution time per blocksize for image inversion

iteration	32	96	192	193	1024
1	1,940416	0,914080	0,452640	0,925472	0,848608
2		0,68192	0,381952	0,825696	
3		0,742240			
4			0,311328	0,725728	
6			0,308224		
8			0,296032		

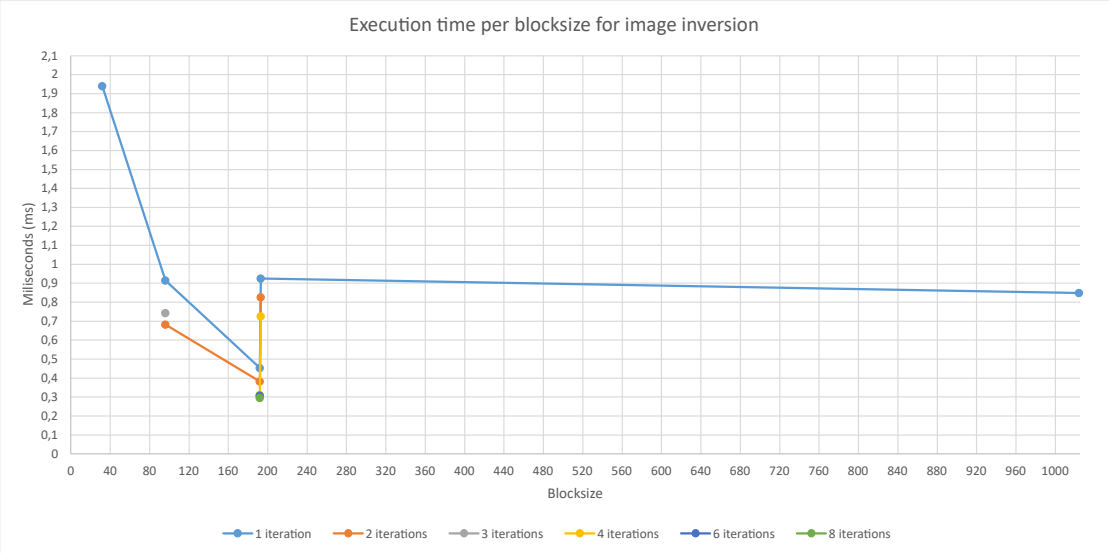


Figure 1: Execution time per blocksize for image inversion

Table 2: Execution time per blocksize for image colouring

iteration	32	96	192	1024
1	2,288128	1,416640	1,378848	1,514976
2	1,832832	1,312064	1,278048	1,368000
4		1,319904	1,315360	1,324608
6		1,298688	1,290752	
8	1,546400	1,269984	1,276000	1,271744
10		1,253120	1,269408	1,287744
16	1,528864	1,236352	1,254016	1,230336



Figure 2: Execution time per blocksize for image colouring